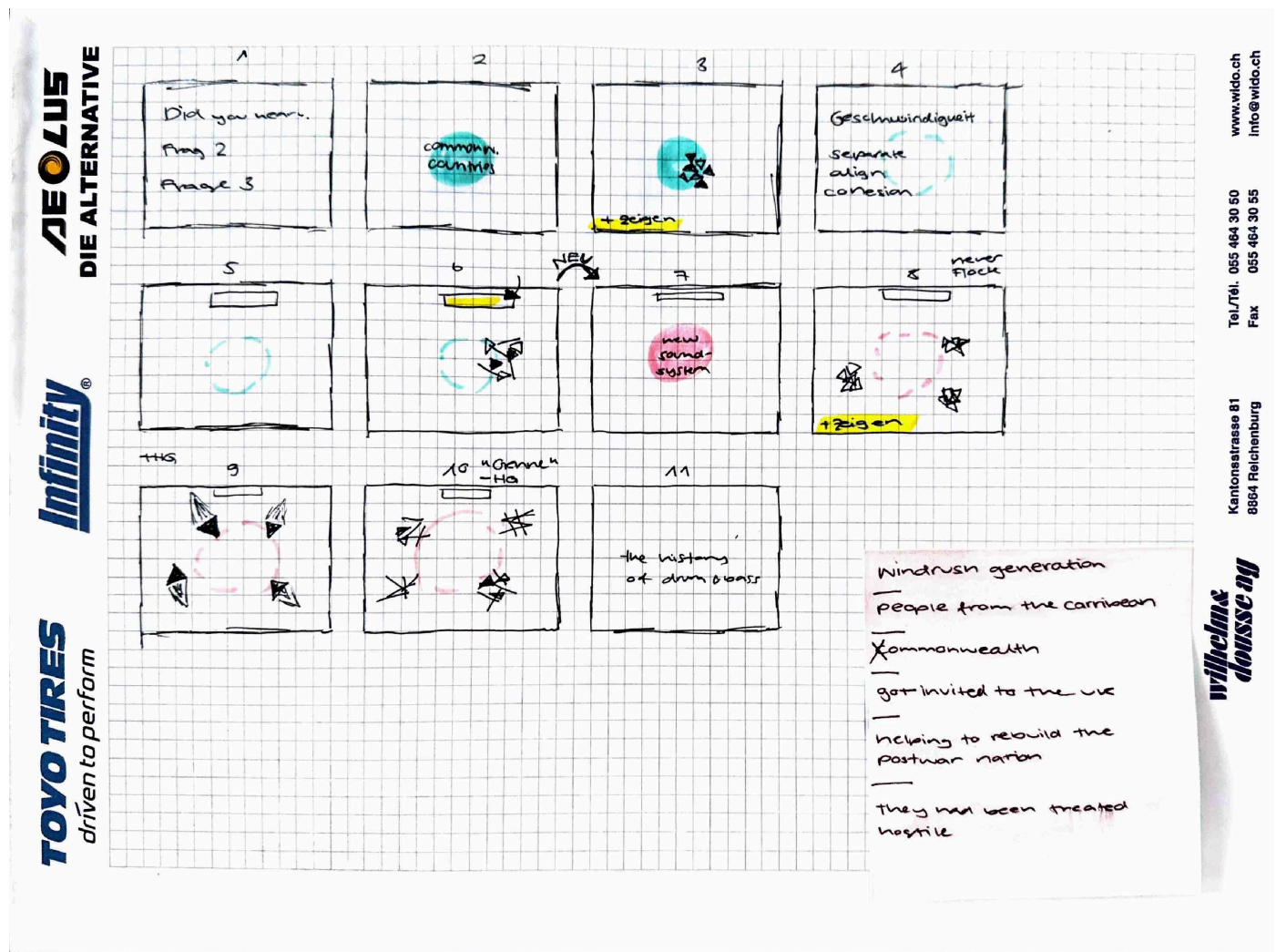


{creative} [coding]

Finale Präsentation

Storyboard



Code

```
// {"P5LIVE":{"name":"first_scene_001","mod":1780240676090}}

let meAskYou = ["The history of drum and bass and the windrush generation.",
"Did you hear about the windrush generation before?",
"What does it have to do with drum and bass?",
"And how was drum and bass being formed in the UK? "]
let questions = meAskYou.join(' ')

function setup() {
  createCanvas(windowWidth, windowHeight);
}
```

```

function draw() {
  // live wechselt zwischen 0 und 7 – steuert das textLeading (Zeilenabstand)
  let live = (frameCount%6)
  // frameRate: 2 fps → langsamer Wechsel, gut für Lesbarkeit
  frameRate(2);

  background(0, 0, 255, 150);
  fill(255)
  textSize(60)
  textWrap(WORD)
  textFont("Satoshi")
  textLeading(32*live);
  textStyle(random([ITALIC,NORMAL]))
  text(questions.repeat(random(16)),100,10,windowWidth/1.1,windowHeight);
}

// noprotect

// -----
// Flocking
// Simuliert Schwarmverhalten nach Craig Reynolds (1986):
// jeder Boid folgt drei einfachen Regeln:
//   Separation – Abstand zu Nachbarn halten
//   Alignment – Richtung der Nachbarn angleichen
//   Cohesion – Zur Mitte der Gruppe bewegen
// -----

// Farbpalette für die Wörter – jedes Wort bekommt eine feste Farbe
const PALETTE = [
  "#ffffff", // reines Weiss
  "#ff6600", // Jungle-Orange
  "#ffcc00", // Goldgelb – Metalheadz
  "#00ff99", // Acid-Grün
  "#ff0066", // Neon-Pink
  "#ffe44d", // helles Amber – statt Lila
  "#ff3300", // Blutrot
  "#f0f000", // Neon-Gelb – statt Elektrisch-Blau
];

// Wörterbuch: speichert welches Wort welche Farbe hat
// damit gleiche Wörter immer dieselbe Farbe bekommen
const wordColorMap = {};
let colorIndex = 0;

// Gibt die Farbe eines Wortes zurück / oder Neue
function getWordColor(word) {
  const key = word.toLowerCase();
  if (!wordColorMap[key]) {
    wordColorMap[key] = PALETTE[colorIndex++ % PALETTE.length];
  }
  return wordColorMap[key];
}

// Pool der möglichen Wörter/Symbole die ein Boid anzeigen kann
let wordPool = ["△", "▲"]

```

```

//let wordPool=["△", "▲", "Reggae", "Dub", "Hip-Hop", "Electro Funk", "Soul", "Dancehall", "Grime",
"Punk", "Jungle", "Breakbeat"]
let wordInput;
let flock;

// setup() für den Flocking-Modus inkl. Audio
function setup() {
  createCanvas(windowWidth, windowHeight);
  setupAudio(true);
  a5.ease = .15;

  // Schwarm erstellen und 50 Boids in der Bildschirmmitte starten
  flock = new Flock();
  // ----- ► GEÄNDERT!! -----
  for (let i = 0; i < 50; i++)
    flock.add(new Boid(width/2, height/2));

  ///Eingabefeld für neue Wörter (aktuell deaktiviert)
  //wordInput = createInput('');
  //wordInput.position(width/2 - 140, 12);
  //wordInput.size(280, 30);
  //wordInput.attribute('placeholder', 'Wort eingeben, Enter = hinzufügen');
  //wordInput.elt.addEventListener('keydown', e => {
  //  if (e.key === 'Enter') {
  //    const words = wordInput.value().trim().split(/\s+/).filter(w => w.length > 0);
  //    words.forEach(w => {
  //      if (!wordPool.includes(w)) wordPool.push(w);
  //      getWordColor(w);
  //      flock.add(new Boid(width/2, height/2));
  //    });
  //    flock.boids.forEach(b => {
  //      b.word = wordPool[floor(random(wordPool.length))];
  //    });
  //    wordInput.value('');
  //  }
  //});
}

// -----
// Audioreaktivität
// -----

function draw() {
  // FFT-Analyse aktualisieren – muss jeden Frame aufgerufen werden
  updateAudio();

  // Bass: Durchschnitt der tiefsten FFT-Bins (Index 0–2), mit Faktor 2 verstärkt
  // ► --> Faktor anpassen wenn Bass zu schwach reagiert
  let bass = (2*(fftEase[0] + fftEase[1] + fftEase[2]) / 3);

  // bgAlpha wäre für transparenten Hintergrund
  let bgAlpha = map(bass, 0, 255, 10, 80);
  background(0, 30, 255);
  //background(0, 30, 255, 10);

  // Transparenz und Kreisgrösse reagieren auf Bass und Höhen
  let alpha1 = map(bass, 0, 255, 30, 255);

```

```

let high = (fftEase[10] + fftEase[11] + fftEase[12]) / 3; // Höhen: Index 10–12
let circleSize = map(high, 0, 255, 380, 520); // Kreisgrösse pulsiert mit Höhen

// -----
// Kreise im Hintergrund
// -----

// // 1. Blauer Kreis – reagiert auf Bass
// let circleAlpha = map(bass, 0, 255, 30, 255);

// fill(0, 16, 120, circleAlpha);
// noStroke();
// ellipse(width / 2, height / 2, 300, 300);

// fill(255, circleAlpha);
// textAlign(CENTER, CENTER);
// textWrap(WORD)
// textSize(20);
// textStyle(BOLD);
// text("", width / 2, height / 2);
// //commonwealth countries

// // 2. Mittlerer Kreis – reagiert auf Höhen mit Pulsieren & Transparenz
// fill(255, 255, 255, alphas);
// noStroke();
// ellipse(width / 2, height / 2, circleSize, circleSize);

// fill(255);
// textAlign(CENTER, CENTER);
// textSize(20);
// textStyle(BOLD);
// text("", width / 2, height / 2);
// // new sound-sytem

// -----

// Mausposition steuert Alignment (y-Achse) und Cohesion (x-Achse) des Schwarms
// map() übersetzt Pixel-Pos in einen kleinen Wertebereich (fast 0 bis max)
let x = map(mouseX, 0, width, 0.00001, 6); // Cohesion-Stärke
let y = map(mouseY, 0, height, 0.00001, 1); // Alignment-Stärke

// Bass steuert Separation – bei starkem Beat weichen Boids stärker auseinander
// ----- ■ GEÄNDERT!! -----
// ----- 1.5 --> 4.0 (höherer Wert = mehr Ausweichen)
let separation = map(bass, 0, 255, 0.8, 1.5);
flock.run(separation, y, x);
}

// Neuen Boid an der Mausposition hinzufügen beim Ziehen
function mouseDragged() {
  flock.add(new Boid(mouseX, mouseY));
}

// -----
// Klasse Flock – der Schwarm
// Verwaltet alle Boids und lässt sie jeden Frame updaten

```

```
// -----
class Flock {
  constructor() { this.boids = []; }
  add(b) { this.boids.push(b); }
  // s = separation, a = alignment, c = cohesion
  run(s, a, c) {
    this.boids.forEach(b => b.run(this.boids, s, a, c));
  }
}

// -----
// Klasse Boid – ein einzelnes Tier im Schwarm
// -----
class Boid {
  constructor(x, y) {
    this.pos = createVector(x, y); // Position
    this.vel = createVector(random(-1,1), random(-1,1)); // Startgeschwindigkeit zufällig
    this.acc = createVector(0, 0); // Beschleunigung (wird jeden Frame neu berechnet)

    // ----- ► GEÄNDERT!! -----
    // ----- Boid Properties
    this.r = 1; // Radius: 1 --> 5
    this.ms = 4; // max speed: 4 --> 15
    this.mf = 0.03; // max force: 0.03 --> 1.833
    // Zufälliges Wort aus dem Pool beim Erstellen zuweisen
    this.word = wordPool[floor(random(wordPool.length))];
    // Jedem Boid einen eigenen FFT-Index zuweisen → verschiedene Frequenzen steuern verschiedene
Boids
    this.fftIndex = floor(random(fftEase.length));
  }

  // Wird jeden Frame aufgerufen: Kräfte berechnen, Position updaten, zeichnen
  // ----- ► GEÄNDERT!! -----
  // ----- Direction of Motion
  run(bs, s, a, c) {
    this.acc.add(this.separate(bs).mult(1.2)); // Separation-Kraft: 1.2 --> .mult(s)
    this.acc.add(this.align(bs).mult(1)); // Alignment-Kraft: 1 --> a
    this.acc.add(this.cohesion(bs).mult(1)); // Cohesion-Kraft: 1 --> c
    this.vel.add(this.acc).limit(this.ms); // Geschwindigkeit begrenzen
    this.pos.add(this.vel); // Position updaten

    this.acc.set(0, 0); // Beschleunigung zurücksetzen für nächsten Frame
    this.borders(); // Bildschirmränder prüfen
    this.render(); // Zeichnen
  }

  // Hilfsfunktion: iteriert über alle Boids im Radius dist
  // und wendet fn() auf jeden Nachbarn an – gibt Durchschnitt zurück
  steer(bs, dist, fn) {
    let sum = createVector(0, 0);
    let count = 0;
    for (let o of bs) {
      let d = p5.Vector.dist(this.pos, o.pos);
      if (d > 0 && d < dist) { fn(sum, o, d); count++; }
    }
    return count > 0 ? sum.div(count) : createVector(0, 0);
  }
}
```

```

// Berechnet Lenkkraft: Wunschgeschwindigkeit minus aktuelle Geschwindigkeit, begrenzt auf mf
limit(s) {
  return s.normalize().mult(this.ms).sub(this.vel).limit(this.mf);
}

// Lenkt den Boid in Richtung eines Zielpunkts t
seek(t) {
  return this.limit(p5.Vector.sub(t, this.pos));
}

// SEPARATION: Maintain distance from neighbors
// ----- ■ GEÄNDERT!! -----
// 20 --> 125 (kompakt --> verteilt)
separate(bs) {
  let s = this.steer(bs, 20, (sum, o, d) =>
    sum.add(p5.Vector.sub(this.pos, o.pos).normalize().div(d))); // Je näher, desto stärker
  return s.mag() > 0 ? this.limit(s) : s;
}

// ALIGNMENT: Align with the direction of neighbors
// 50 --> 150 (nur nahe Nachbarn --> gross = gleichmässig)
align(bs) {
  let s = this.steer(bs, 50, (sum, o) => sum.add(o.vel));
  return s.mag() > 0 ? this.limit(s) : s;
}

// COHESION: Move toward the center of the group
// 100 --> 150 (nahe Nachbarn --> hält stark zusammen)
cohesion(bs) {
  let s = this.steer(bs, 100, (sum, o) => sum.add(o.pos));
  return s.mag() > 0 ? this.seek(s) : s;
}

// Teleportiert den Boid auf die gegenüberliegende Seite wenn er den Rand überschreitet
borders() {
  for (let [ax, max] of [['x', width], ['y', height]]) {
    if (this.pos[ax] < -this.r) this.pos[ax] = max + this.r;
    if (this.pos[ax] > max + this.r) this.pos[ax] = -this.r;
  }
}

// -----
// Den Boid als Wort zeichnen, das in Flugrichtung zeigt.
// Gleiche Wörter -> gleiche Farbe (aus wordColorMap)
// -----
render() {
  let size = 14; // fixe Schriftgrösse

  // Zum Aktivieren: Kommentarsymbole entfernen
  // push();
  // translate(this.pos.x, this.pos.y);
  // rotate(this.vel.heading() + radians(90)); // In Flugrichtung drehen
  // fill(getWordColor(this.word));
  // noStroke();
  // textAlign(CENTER, CENTER);
  // textSize(size); // Schriftgrösse proportional zu r + FFT
  // text(this.word, 0, 0);
  // pop();

```

```
}  
}
```

Live-Performance

mit Beispiel-Audio, da ich das Audio von Alina nicht in voller Länge besitze.